

Precharge Design Report

Rutgers Formula Racing

Jeevan Shah
August 13, 2026



Contents

1	Revision Log	2
2	Purpose	2
3	Parties Involved	2
4	Research	2
4.1	Relevant Rules	2
4.2	Why Do We Need This?	2
4.3	Pros & Cons	3
4.4	Design Impact	3
5	Notes/Finding	3
5.1	Core Requirements	3
5.2	Part Choices and Justification	3
5.3	Schematic	4
5.4	Firmware	4
5.4.1	main.zig	4
5.4.2	root.zig	7
6	Conclusion	9

1 Revision Log

Editor	Date	Revision
Jeevan Shah	Aug. 12 2026	1

2 Purpose

Investigate and plan the switch to a microcontroller based precharge system for the 2027 FSAE season.

3 Parties Involved

Name	Team Role	Report Role
Jeevan Shah	Electronics co-Lead & Software Lead	Author

4 Research

4.1 Relevant Rules

- **EV.5.6.1** The Accumulator must contain a Precharge Circuit. The Precharge Circuit must:
 - (a) Be able to charge the Intermediate Circuit to minimum 90% of the Accumulator voltage before closing the second IR.
 - (b) Be supplied from the Shutdown Circuit EV.7.1.
- **EV.5.6.2** The Intermediate Circuit must precharge before closing the second IR.
 - (a) The end of precharging must be controlled by feedback by monitoring the voltage in the Intermediate Circuit.
- **EV.5.6.3** The Tractive System must contain a Discharge Circuit. The Discharge Circuit must be:
 - (a) Wired in a way that it is always active when the Shutdown Circuit is open.
 - (b) Able to discharge the Intermediate Circuit capacitors if the MSD has been opened.
 - (c) Not be fused.
 - (d) Designed to handle the maximum Tractive System voltage for minimum 15 seconds.
- **EV.5.6.4** Positive Temperature Coefficient (PTC) devices must not be used to limit current for the Precharge Circuit or Discharge Circuit.
- **EV.5.6.5** The precharge relay must be a mechanical type relay.
- **EV.6.5.7** Voltage Separation
 - (a) HV and GLV must be separated galvanically. If on same board, spacing requirements must be met.

4.2 Why Do We Need This?

The RFR '26 season came to end when E-meter data was pulled during EV-active revealing that our precharge was not functional and instead closing both isolation relays (IRs) at the same time. Extensive testing and debugging of both RFR '26's and RFR 25's precharge board revealed that there seemed to be a fundamental issue in our design as when both board were supplied with high voltages and high capacitance (i.e. tested in the car), both IRs still shut instantly. Switching to a microcontroller based system allows us to read both the inverter and battery management system (BMS) voltages via CAN which, when coupled with basic software logic, helps ensure that the IRs will never close early.

4.3 Pros & Cons

Pros	Cons
1. A microcontroller based system is very easy to change and modify compared to an analog system	1. Team members must know how to code and work with embedded systems to work and troubleshoot
2. Using software logic to regulate the IRs ensures that they will never close early	2. Changing our precharge design may result in difficulties involved with passing ESF
3. The microcontroller implementation iterates off of RFR '26's precharge design, keeping the high voltage section which has been proven to work	

4.4 Design Impact

The size of the PCB will not change, which means currently implemented packaging solutions will hold valid for the RFR '27 season. Any weight changes are non-negligible due to the an about equivalent amount of components swapped from the previous design.

5 Notes/Finding

5.1 Core Requirements

The core functionality of the precharge circuit is

1. to be powered off of the shutdown circuit (SDC)
2. close the first IR as soon as the SDC is closed
3. close the second IR once the inverter voltage is (at least) 90% of the total BMS voltage at any given state of charge (SOC)
 - (a) it follows that the precharge must monitor the BMS and inverter voltage

5.2 Part Choices and Justification

Because we are iterating on a previous design it is known that our precharge will function off of 24V from the shutdown circuit (SDC). The SHV24-1A85-78D3K reed relay from Standex-Meder electronics will be reused as it is proven to work in two other designs and past EVs, has previously been granted ESF approval, and has a reasonably low 288mW power consumption. We will also utilize the STM32F103T8U6 microcontroller from ST electronics since the STM32F series was utilized in RFR '26 with the successful implementation of a custom thermistor module. The STM32F103T8U6 was chosen in specific because it is the most basic microcontroller from ST that has CAN communication capabilities. A more advanced/powerful microcontroller is not needed as the software, general purpose input and output pin (GPIO), and memory requirement are extremely low. The STM32F103T8U6 requires 3.3V as input, which is notably lower than 24V. Because of this, we will utilize the IMA0124S3V3 DCDC converter from XP Power to bring our 24V supply voltage down to 3.3V. The IMA0124S3V3 will also power the non-isolated Texas Instruments CANTransceiver TCAN3413DR, enabling us to read CAN frames from both the inverter and BMS.

5.3 Schematic

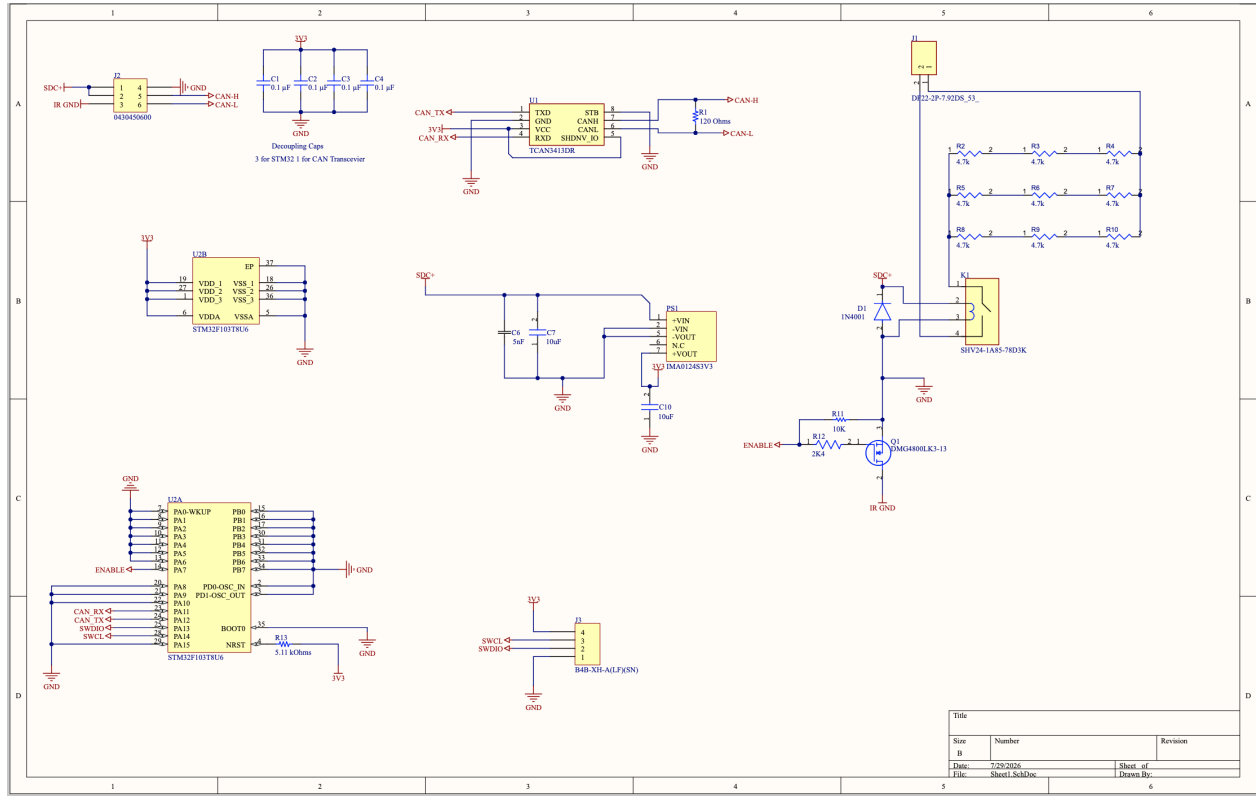


Figure 1: Schematic

The schematic is extremely simple as it only features the microcontroller, DCDC converter, CANTransciever, and the high voltage relay. We are taking advantage of a GPIO pin (labeled 'ENABLE') to switch the MOSFET. As mentioned earlier the IMA0124S3V3 is used to convert the 24V SDC supply into a usable 3.3V for the microcontroller and CANTransciever. The relay section (A5, B5) remains mostly identical from the previous design, save for the fact that the third input is no longer needed as voltages are no longer being scaled. The current draw of the STM32 along with the CANTransciever combined is about 110mA, even with the additional decoupling capacitors the 300mA rating of the DCDC converter should never be exceeded.

5.4 Firmware

The embedded code that run on the microcontroller is written in Zig - a low level language that works as an alternative to C while providing no hidden overloads. This is a major benefit to working in Zig as the code you read is exactly what is happening. Structurally, the code is split into two files: `main.zig` and `root.zig`. `root.zig` serves as the abstract class definitions while `main.zig` contains the actual implementation as per our setup. For interfacing with the STM32, we utilized the `microzig` library.

5.4.1 main.zig

The `main.zig` file is extremely simple. Starting at the top, there are the basic imports and setup functions.

```
1 const std = @import("std");
2 const microzig = @import("microzig");
3 const logic = @import("root.zig");
4 const stm32 = microzig.hal;
```

```
5 const peripherals = microzig.chip.peripherals;
6 const rcc = stm32.rcc;
7 const gpio = stm32.gpio;
8 const time = stm32.time;
9
10 pub const microzig_options: microzig.Options = .{ .interrupts = .{} };
11 pub const panic = microzig.panic;
12 pub const std_options = microzig.std_options(.{});
13
14 comptime {
15     _ = microzig.export_startup();
16 }
17
18 const protocol = logic.base_config;
19 const loop_period_ms: u32 = 5;
20
21 const pins = struct {
22     const enable = gpio.Pin.from_port(.A, 7);
23     const can_rx = gpio.Pin.from_port(.A, 11);
24     const can_tx = gpio.Pin.from_port(.A, 12);
25 };
26
27 fn systemClockConfig() !void {
28     _ = try rcc.apply(.{
29         .SYSCLKSource = .PLL1_P,
30         .PLLSourceVirtual = .HSE_Div_PREDIV,
31         .PLLMUL = .Mul9,
32         .AHBCLKDivider = .Div1,
33         .APB1CLKDivider = .Div2,
34         .ADCPresc = .Div6,
35         .flags = .{ .HSE0scillator = true },
36     });
37 }
```

Following the imports and setup comes the CAN configuration for the STM32. The `packedFilterID()` function converts a logical CAN ID into the bit layout required by the STM32 `bxCAN` acceptance-filter registers. This is then used on lines 69 and 70 to configure `FIF00` (the CAN mailbox queue) to only allow the BMS and inverter CAN frames to be read.

```
38 fn packedFilterId(spec: logic.VoltageSpec) u32 {
39     return switch (spec.kind) {
40         .extended => ((spec.id & 0x1FFF_FFFF) << 3) | (1 << 2),
41         .standard => (spec.id & 0x7FF) << 21,
42     };
43 }
44
45 fn canInit() void {
46     const can = peripherals.CAN;
47
48     // Enter initialization mode before changing bit timing.
49     can.MCR.modify(.{ .SLEEP = 0, .INRQ = 1 });
50     while (can.MSR.read().INAK == 0) {}
51
52     // APB1 is 36 MHz. BRP=4, 18 time quanta/bit gives 500 kbit/s;
53     // register fields contain each value minus one.
54     if (protocol.bitrate != 500_000)
55         @compileError("Update bxCAN bit timing before selecting another bitrate");
56     can.BTR.modify(.{
57         .BRP = 3,
```

```

58     .@"TS[0]" = 14,
59     .@"TS[1]" = 1,
60     .SJW = 0,
61     .LBKM = 0,
62     .SILM = .Normal,
63 };
64
65 can.FMR.modify(.{ .FINIT = 1 });
66 can.FM1R.raw |= 1; // bank 0, identifier-list mode
67 can.FS1R.raw |= 1; // bank 0, 32-bit scale
68 can.FFA1R.raw &= ~@as(u32, 1); // bank 0 -> FIFO0
69 can.FB[0].FR1.raw = packedFilterId(protocol.bms);
70 can.FB[0].FR2.raw = packedFilterId(protocol.inverter);
71 can.FA1R.raw |= 1;
72 can.FMR.modify(.{ .FINIT = 0 });
73
74 can.MCR.modify(.{ .INRQ = 0 });
75 while (can.MSR.read().INAK == 1) {}
76 }
77
78 fn canReceive() ?logic.Frame {
79     const can = peripherals.CAN;
80     if (can.RFR[0].read().FMP == 0) return null;
81
82     const fifo = &can.RX[0];
83     const rir = fifo.RIR.read();
84     var frame = logic.Frame{
85         .id = undefined,
86         .kind = if (rir.IDE == .Extended) .extended else .standard,
87         .remote = rir.RTR == .Remote,
88         .dlc = fifo.RDTR.read().DLC,
89         .data = undefined,
90     };
91     frame.id = switch (frame.kind) {
92         .extended => (@as(u32, rir.STID) << 18) | rir.EXID,
93         .standard => rir.STID,
94     };
95     std.mem.writeInt(u32, frame.data[0..4], fifo.RDLR.raw, .little);
96     std.mem.writeInt(u32, frame.data[4..8], fifo.RDHR.raw, .little);
97
98     can.RFR[0].modify(.{ .RFOM = 1 });
99     return frame;
100 }

```

Following the CAN setup is the `main()` function which contains the master superloop for the program. The first ten lines contain the clock setup, pin setup, and initialization of the timer variable and the primary controller.

```

101 pub fn main() !void {
102     try systemClockConfig();
103
104     rcc.enable_clock(.GPIOA);
105     rcc.enable_clock(.AFIO);
106     rcc.enable_clock(.CAN);
107     rcc.enable_clock(.TIM2);
108
109     // Establish the safe output state before initializing communications.
110     pins.enable.put(0);
111     pins.enable.set_output_mode(.general_purpose_push_pull, .max_2MHz);

```

```

112 pins.can_rx.set_input_mode(.floating);
113 pins.can_tx.set_output_mode(.alternate_function_push_pull, .max_50MHz);
114
115 time.init_timer(.TIM2);
116 canInit();
117
118 var now_ms: u32 = 0;
119 var controller = logic.Controller.init(protocol, now_ms);

```

Entering into the primary superloop, there are four main sections. Lines 120 – 121 contains the entry to the superloop and unpacks CAN frames.

```

120 while (true) {
121     while (canReceive()) |frame| controller.ingest(frame, now_ms);

```

Next, the `tick()` function is called which updates the `state` variable.

```

122     controller.tick(now_ms);

```

Then, the program checks the updated state. If precharging is complete, then the enable pin is set to high, and an infinite loop is entered. If there was a fault, then the enable pin stays low and an infinite loop is entered. In any of the other states (`waiting_for_bms` or `precharging`) no further action is taken.

```

123     switch (controller.state) {
124         .complete => {
125             pins.enable.put(1);
126             while (true) time.sleep_ms(1_000);
127         },
128         .fault_latched => {
129             pins.enable.put(0);
130             while (true) time.sleep_ms(1_000);
131         },
132         else => {},
133     }

```

Finally, there is a brief delay before the next cycle is entered to ensure that there is necessary time for the CAN mailbox to be flushed, and the timer variable `now_ms` is updated.

```

134         time.sleep_ms(loop_period_ms);
135         now_ms += loop_period_ms; // leverage wrapping addition to keep the timer
136         }
137     }

```

5.4.2 root.zig

As mentioned earlier, `root.zig` contains the abstract `Controller` class which is used in `main.zig`. Within `root.zig` are multiple enum and const definitions, but for the purpose of this document I will simply be reviewing the primary functions used.

The `ingest()` function (see `main.zig:121`), comes from `root.zig`. At its core, `ingest()` reads a CAN frame and decides if it is from the BMS or the inverter. For either source, the program will convert the CAN frame into a voltage via the `decodeVoltage()` function and then the voltage is updated along with the timestamp. If the frame is from the BMS then there is a check to update the state from `waiting_for_bms` to `precharge` which happens when the first BMS frame is received as `waiting_for_bms` is the default state. If the frame is from the inverter then the `qualify()` function is called.

```

107 pub fn ingest(self: *Controller, frame: Frame, now_ms: u32) void {
108     if (self.state == .complete or self.state == .fault_latched) return;
109

```



```

110     if (matches(frame, self.config.bms)) {
111         const value = decodeVoltage(frame, self.config.bms) catch {
112             self.latch(.malformed_frame);
113             return;
114         };
115         if (!self.plausible(value)) return self.latch(.implausible_voltage);
116         self.bms = .{ .value = value, .timestamp_ms = now_ms };
117         if (self.state == .waiting_for_bms) {
118             self.state = .precharging;
119             self.precharging_started_ms = now_ms;
120         }
121     } else if (matches(frame, self.config.inverter)) {
122         const value = decodeVoltage(frame, self.config.inverter) catch {
123             self.latch(.malformed_frame);
124             return;
125         };
126         if (!self.plausible(value)) return self.latch(.implausible_voltage);
127         self.inverter = .{ .value = value, .timestamp_ms = now_ms };
128         self.qualify(now_ms);
129     }
130 }

```

The `qualify()` function is where the actual logic lives to decide whether or not to close the IR. Up to line 165 is basic setup and fault checking. Lines 155 – 158 checks if a BMS voltage exists, and if not it sets the amount of consecutive qualifying samples to 0 and returns early. Next, lines 160 – 163 check to ensure that the most recent BMS voltage reading is less than a defined threshold (`freshness_timeout_ms`), if it isn't, a fault is thrown. Finally, lines 165 – 173 check whether the inverter voltage is greater than 90% of the BMS voltage. If it is, the `consecutive_qualifying` variable is increased. After `qualifying_samples` amount of valid consecutive frames, precharging is complete.

```

154 fn qualify(self: *Controller, now_ms: u32) void {
155     const bms = self.bms orelse {
156         self.consecutive_qualifying = 0;
157         return;
158     };
159     const inverter = self.inverter.?.;
160     if (elapsed(now_ms, bms.timestamp_ms) >= self.config.freshness_timeout_ms) {
161         self.latch(.stale_data);
162         return;
163     }
164
165     const lhs = @as(u64, inverter.value) * 100;
166     const rhs = @as(u64, bms.value) * self.config.threshold_percent;
167     if (lhs >= rhs) {
168         self.consecutive_qualifying += 1; // saturating addition prevents
169         wrapping to zero
170         if (self.consecutive_qualifying >= self.config.qualifying_samples)
171             self.state = .complete;
172     } else {
173         self.consecutive_qualifying = 0;
174     }
175 }

```

The final primary function in `root.zig` is the `tick()` function. The `tick()` function's primary purpose is to ensure that there is no stale data. The function ensures that the entire precharge cycle isn't taking too long (lines 134 – 135), the BMS has sent its first frame within `freshness_timeout_ms` (lines 137 – 141), the most recent BMS and inverter frames are within `freshness_timeout_ms` (lines 143 – 148), and that the time for the inverter voltage to reach 90% of the BMS voltage is within `freshness_timeout_ms`.

```

132 pub fn tick(self: *Controller, now_ms: u32) void {
133     if (self.state == .complete or self.state == .fault_latched) return;
134     if (elapsed(now_ms, self.started_ms) >= self.config.precharge_timeout_ms)
135         return self.latch(.timeout);
136
137     if (self.state == .waiting_for_bms) {
138         if (elapsed(now_ms, self.started_ms) >= self.config.freshness_timeout_ms)
139             self.latch(.stale_data);
140         return;
141     }
142
143     const bms = self.bms or else return self.latch(.stale_data);
144     if (elapsed(now_ms, bms.timestamp_ms) >= self.config.freshness_timeout_ms)
145         return self.latch(.stale_data);
146     if (self.inverter) |reading| {
147         if (elapsed(now_ms, reading.timestamp_ms) >= self.config.
148             freshness_timeout_ms)
149             return self.latch(.stale_data);
150     } else if (elapsed(now_ms, self.precharging_started_ms.?) >= self.config.
151         freshness_timeout_ms) {
152         return self.latch(.stale_data);
153     }
154 }

```

The default config can be seen below.

```

1 pub const ProtocolConfig = struct {
2     ready: bool,
3     bitrate: u32,
4     bms: VoltageSpec,
5     inverter: VoltageSpec,
6     min_voltage: u32 = 1,
7     max_voltage: u32 = 390,
8     threshold_percent: u8 = 90,
9     qualifying_samples: u8 = 1,
10    freshness_timeout_ms: u32 = 5_000,
11    precharge_timeout_ms: u32 = 300_000,
12 };

```

View the entire codebase on the [Rutgers Formula Racing Github](#) for more.

6 Conclusion

Switching to a microcontroller based precharge system allows us to implement something that is more reliable and easier to modify than the analog alternative.